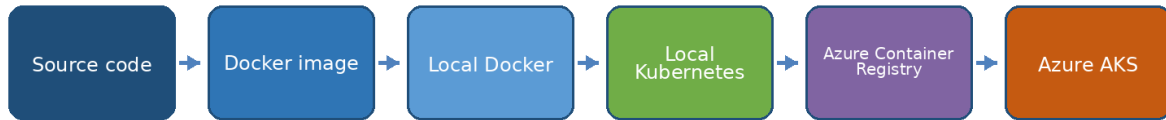


Practical Name: Deploying application through kubernetes to Azure cloud. to Azure cloud.

Windows 11 PowerShell edition

One application moving through three environments



The same small application moving from code to cloud

This practical teaches the complete path from a small Python web application to a container, a local Kubernetes deployment, and an Azure Kubernetes Service deployment using the Azure portal or PowerShell. Students also update the application and observe how Kubernetes performs a controlled rolling update.

Project name Hello Cloud App. The application returns a small JSON response and a health response so that students can easily see when a new version is running.

Important Azure note Azure for Students is credit based. Check your remaining credit before creating AKS, monitor usage, and delete the resource group at the end of the lab. The Azure section includes visual Azure portal steps and an optional PowerShell route.

Practical Overview

By the end of this practical, a student will be able to create a small Flask application, build it into a Docker image, test it on Windows 11, run it with local Kubernetes, publish the image to Azure Container Registry, deploy it to Azure Kubernetes Service, update the running application, and roll back when necessary.

Stage	Main tool	Evidence to collect
Create	Python and Flask	Browser response from localhost
Package	Docker Desktop	docker ps and running container
Orchestrate locally	Docker Desktop Kubernetes	Pods and Service output
Publish	Azure portal and ACR	Successful image push
Deploy in cloud	Azure portal and AKS	External IP and browser response
Update	Portal or kubectl	Version change and rollout status

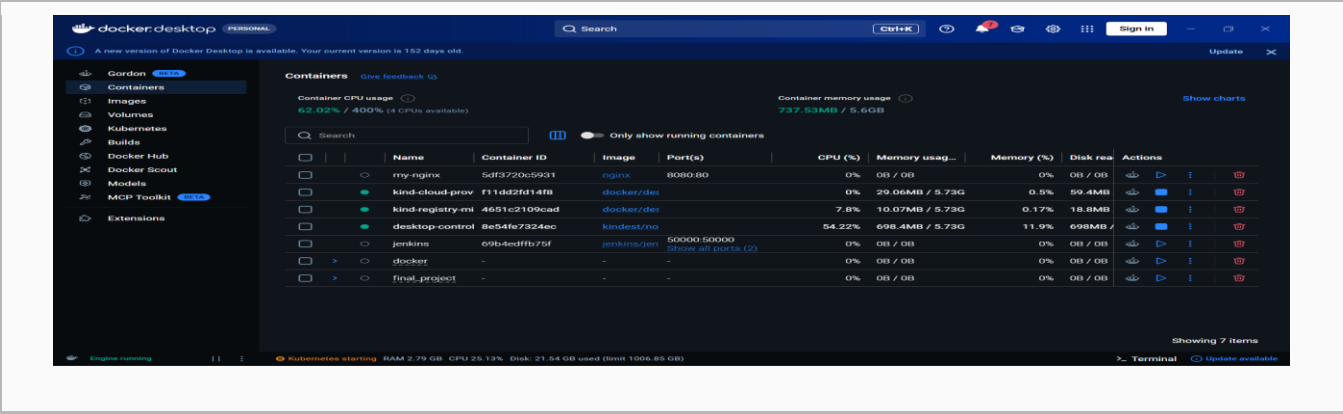
Learning Outcomes

- Explain the difference between source code, a Docker image, a container, a pod, a deployment, and a service.
- Run the application locally before introducing Kubernetes.
- Use a Kubernetes Deployment to keep two or more application replicas running.
- Use a Kubernetes Service to provide a stable address for changing pods.
- Use Azure Container Registry to store a private container image and AKS to run it.
- Perform a version update, observe a rolling update, and practise rollback.
- Use the Azure portal to create resources, apply a Kubernetes YAML manifest, observe workloads, update the image, and clean up the resource group.

What Students Need

- Windows 11 computer with Docker Desktop installed and running.
- PowerShell or Windows Terminal.
- Visual Studio Code or another text editor.
- An Azure for Students subscription with an available credit balance.
- Internet access for downloading the base image and pushing the image to Azure.

Screenshot space 1 Record the tools installed on the computer



Step 1 Prepare the Windows Machine

Open PowerShell after starting Docker Desktop. Run the following commands. If a command is not recognised, install the missing tool and open a new PowerShell window.

PowerShell

```
docker --version
kubectl version --client
az --version
```

Docker Desktop provides Docker Engine and can provide a local Kubernetes cluster. Azure CLI is used to sign in, create Azure resources, push the image to ACR, and connect kubectl to AKS.

Tool	Purpose in this practical
Docker Desktop	Build images and run containers locally
kubectl	Send commands to a Kubernetes cluster
Azure CLI	Create and manage Azure resources
PowerShell	Run the commands on Windows 11

If Azure CLI is not installed, use the official Windows installer or Azure Cloud Shell. If kubectl is not available, enable Kubernetes in Docker Desktop and restart the terminal.

Useful Installation Links

[Install Azure CLI on Windows](#)

[Enable Kubernetes in Docker Desktop](#)

[Azure for Students](#)

Screenshot space 2 Show docker kubectl and az version output

```
C:\>docker --version
Docker version 29.4.1, build 055a478

C:\>kubectrl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1

C:\>az --version
azure-cli                2.90.0
core                     2.90.0
telemetry                1.1.0

Dependencies:
msal                     1.36.0
azure-mgmt-resource      24.0.0

Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\Admin\.azure'
Extensions directory 'C:\Users\Admin\.azure\cliextensions'

Python (Windows) 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)]

Legal docs and information: aka.ms/AzureCliLegal

Your CLI is up-to-date.
```

Step 2 Create the Project Folder

Use PowerShell to create a project folder. You can use another folder, but use the same folder consistently for the whole practical.

PowerShell

```
New-Item -ItemType Directory -Path C:\student-cloud-lab -Force
Set-Location C:\student-cloud-lab\code .
```

Create the following files in Visual Studio Code. The complete contents are provided below so that students can copy them exactly.

File	Purpose
app.py	Flask application code
requirements.txt	Python dependency list
Dockerfile	Instructions for building the image
k8s-local.yaml	Local Kubernetes Deployment and Service
k8s-azure.yaml	Azure Kubernetes Deployment and Service

Step 3 Write the Flask Application

The application has two endpoints. The home endpoint shows the version. The health endpoint allows Docker and Kubernetes to check whether the application is responding.

File One Application Code

Python

```
from flask import Flask, jsonify

app = Flask(__name__)
APP_VERSION = "v1"

@app.get("/")
def home():
    return jsonify(
        app="Hello Cloud App",
        version=APP_VERSION,
        message="The application is running"
    )

@app.get("/health")
def health():
    return jsonify(status="healthy")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

File Two Python Dependencies

Text

```
Flask==3.1.0
```

Step 4 Run the App Without Docker

This checkpoint proves that the Python code works before Docker is introduced. Run these commands from the project folder.

PowerShell

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

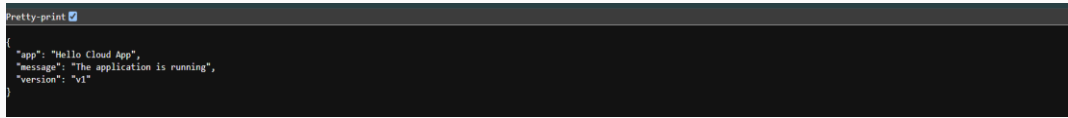
Open these addresses in a browser. The terminal should remain open while the Flask server is running.

Browser addresses

```
http://localhost:5000
http://localhost:5000/health
```

Expected result: the first address shows a JSON response containing version v1. The second address shows a healthy status.

Screenshot space 3 Show the Flask app running directly from Python



Stop the Flask server with Ctrl plus C. Deactivate the virtual environment when you are finished with this checkpoint.

PowerShell

```
deactivate
```

Step 5 Build and Run the Docker Image

A Docker image packages the application and its runtime instructions. A container is a running instance of that image.

File Three Docker Instructions

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000
CMD ["python", "app.py"]
```

Build the Image

PowerShell

```
docker build -t hello-cloud-app:v1 .
docker images
```

The tag has two parts. hello-cloud-app is the image name and v1 is the version tag. Version tags make it easier to update and roll back applications.

Run the Container

PowerShell

```
docker run --name hello-cloud-app -p 5000:5000 hello-cloud-app:v1
docker ps
```

The command maps port 5000 on the Windows machine to port 5000 inside the container. Open the same browser addresses again.

Browser addresses

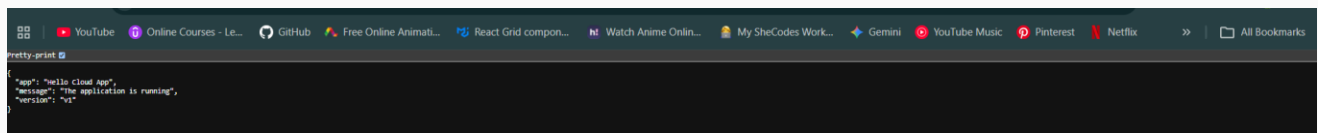
```
http://localhost:5000
http://localhost:5000/health
```

Inspect and Stop the Container

PowerShell

```
docker logs hello-cloud-app
docker stop hello-cloud-app
docker rm hello-cloud-app
```

Screenshot space 4 Show docker build docker ps and the browser response



Common problem If port 5000 is already in use, run the container on another Windows port: `docker run --name hello-cloud-app -p 5050:5000 hello-cloud-app:v1`. Then open `http://localhost:5050`.

Step 6 Enable Local Kubernetes

Docker Desktop can run a small local Kubernetes cluster for learning. This is suitable for classroom practice. It is not a replacement for a production cluster.

1. Open Docker Desktop.
2. Open Settings and then Kubernetes.
3. Enable Kubernetes and choose Apply or Apply and restart.
4. Wait until Docker Desktop reports that Kubernetes is running.

PowerShell

```
kubectl config get-contexts
kubectl config use-context docker-desktop
kubectl get nodes
```

Expected result: one node named docker-desktop with a Ready status.

Screenshot space 5 Show Docker Desktop Kubernetes running and kubectl get nodes

```
C:\student-cloud-lab>kubectl config get-contexts
CURRENT  NAME           CLUSTER    AUTHINFO    NAMESPACE
*         docker-desktop  docker-desktop  docker-desktop

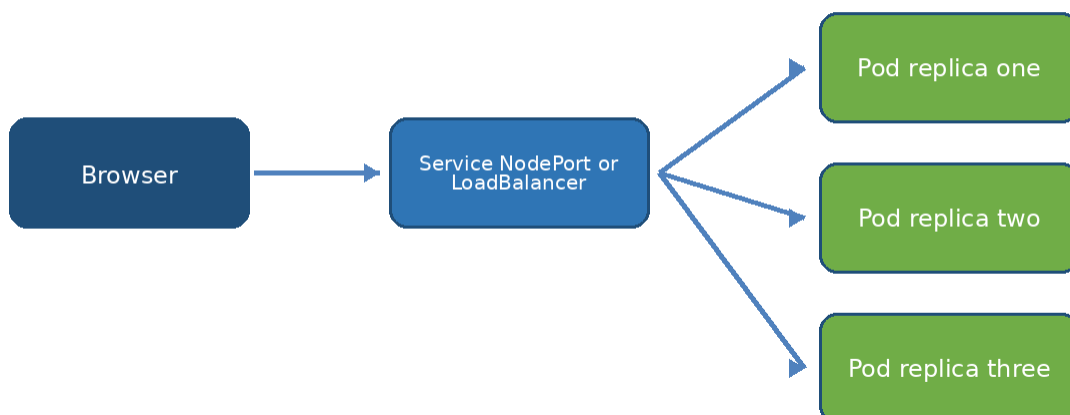
C:\student-cloud-lab>kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
desktop-control-plane  Ready    control-plane  75s   v1.36.4

C:\student-cloud-lab>|
```

Step 7 Deploy the App to Local Kubernetes

The Deployment asks Kubernetes to keep two replicas of the app running. The Service gives the replicas one stable access point. Because the image is already inside Docker Desktop, the local cluster can use it without pulling from a public registry.

Kubernetes keeps the service address stable while pods run the app



A Service sends traffic to healthy pod replicas
From Code to Cloud Student Practical

File Four Local Kubernetes Configuration

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-cloud-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-cloud-app
  template:
    metadata:
      labels:
        app: hello-cloud-app
    spec:
      containers:
        - name: hello-cloud-app
          image: hello-cloud-app:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
              initialDelaySeconds: 3
              periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
              initialDelaySeconds: 10
              periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: hello-cloud-app
spec:
  selector:
    app: hello-cloud-app
  type: NodePort
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

Apply and Check the Deployment

PowerShell

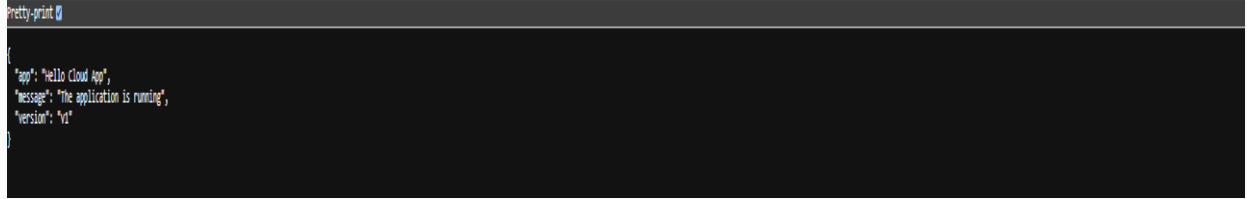
```
kubectl apply -f .\k8s-local.yaml
kubectl rollout status deployment/hello-cloud-app
kubectl get deployments
kubectl get pods -o wide
kubectl get service hello-cloud-app
```

Open the local Kubernetes service in a browser.

Browser addresses

```
http://localhost:30080
http://localhost:30080/health
```

Screenshot space 6 Show two running pods the Service and the browser response



```
pretty-print
{
  "app": "Hello Cloud App",
  "message": "The application is running",
  "version": "v1"
}
```

Scale the Local Deployment

PowerShell

```
kubectl scale deployment hello-cloud-app --replicas=3
kubectl get pods
kubectl get deployment hello-cloud-app
```

Kubernetes creates another pod because the desired state is now three replicas. The Service continues to use the same address.

Step 8 Update the App Locally

The update demonstrates why versioned images and Kubernetes Deployments are useful. Change the message and version in app.py.

Python change

```
APP_VERSION = "v2"

# Replace the message inside home with this value
message="Version two is running"
```

Build a new image with a new tag. Do not overwrite v1. Keeping v1 makes rollback easy.

PowerShell

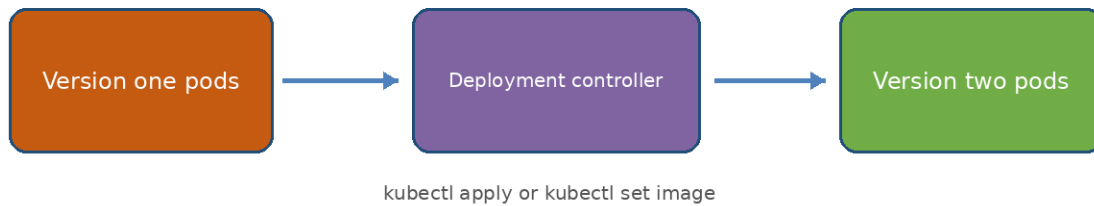
```
docker build -t hello-cloud-app:v2 .
docker images
```

Edit k8s-local.yaml and change only the image line to hello-cloud-app:v2. Then apply the updated configuration.

PowerShell

```
kubectl apply -f .\k8s-local.yaml
kubectl rollout status deployment/hello-cloud-app
kubectl get pods
kubectl get deployment hello-cloud-app -o wide
```

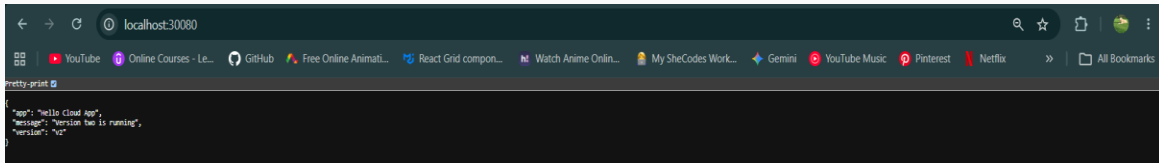
Rolling update: new pods become ready before old pods disappear



A rolling update replaces old pods with new pods in a controlled way

Refresh <http://localhost:30080>. The response should now show v2. During a healthy rolling update, Kubernetes starts replacement pods and waits for readiness before removing old pods.

Screenshot space 7 Show v2 in the browser and rollout status

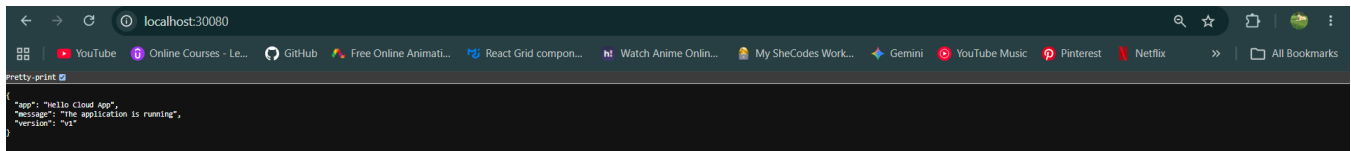


Roll Back the Local Update

PowerShell

```
kubectl rollout history deployment/hello-cloud-app  
kubectl rollout undo deployment/hello-cloud-app  
kubectl rollout status deployment/hello-cloud-app
```

Refresh the browser. The app should return to the previous version. This is one of the main benefits of managing the app as a Kubernetes Deployment.



Step 9 Sign In to Azure

Azure Container Registry stores the image. Azure Kubernetes Service runs the Kubernetes cluster. Use the Azure Student subscription only for the time needed for the practical, then delete the resource group.

Visual Azure Portal Path

Use these browser steps for the sign-in, subscription selection, and resource group creation. The PowerShell blocks that follow are an optional automated route.

1. Open <https://portal.azure.com> and sign in with the account linked to the Azure for Students subscription.
2. In the portal search bar, open Subscriptions. Select the student subscription and confirm that its status is Active. Copy the Subscription ID only if you plan to use the optional PowerShell route.
3. Search for Resource groups, select Create, choose the student subscription, and select Create new. Enter rg-hello-cloud as the resource group name.
4. Choose Central India, or another region available to the subscription, select Review + create, and then select Create.
5. Open the new resource group and keep this portal tab available. The ACR and AKS resources will be created in this group.

Visual checkpoint. Capture the subscription and Resource groups pages for Screenshot space 8.

The portal uses the values below. Record them before moving to the next step.

Optional PowerShell route

```
az login
az account list --output table
az account show --output table
```

If more than one subscription is listed, set the subscription that contains your student credit. Replace the placeholder with the correct subscription ID shown by the previous command.

Optional PowerShell route

```
az account set --subscription <YOUR_SUBSCRIPTION_ID>
az account show --output table
```

Create Variables

Portal field	Value for this practical
Resource group	rg-hello-cloud
Region	Central India, or another region available to the subscription
ACR name	Unique lowercase name, for example hellocloudacr1234
AKS cluster name	hello-cloud-aks
AKS pricing tier	Free, if the portal displays this option

Optional PowerShell route

```
$SUFFIX = Get-Random -Minimum 1000 -Maximum 9999
$RESOURCE_GROUP = "rg-hello-cloud"
$LOCATION = "centralindia"
$ACR_NAME = "hellocloudacr$SUFFIX"
$AKS_NAME = "hello-cloud-aks"

az group create --name $RESOURCE_GROUP --location $LOCATION
```

Region note If Azure reports a capacity or quota problem in centralindia, choose another region available to your subscription, such as eastus, and run the location variable again before creating resources.

Screenshot space 8 Show the Azure subscription and created resource group in the portal

```
C:\student-cloud-lab>az group create --name %RESOURCE_GROUP% --location %LOCATION%
{
  "id": "/subscriptions/e18c6649-2bea-435e-85e8-d6ab55b7237f/resourceGroups/rg-hello-cloud",
  "location": "centralindia",
  "managedBy": null,
  "name": "rg-hello-cloud",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

```
C:\student-cloud-lab>
```

Step 10 Create Azure Container Registry

ACR is a private image registry. The image must be tagged with the registry login server before it can be pushed.

Visual Azure Portal Path

Create the private registry in the browser before pushing the image.

1. In the Azure portal, select Create a resource. Search for Container Registry, then select Create.
2. On Basics, select the student subscription and rg-hello-cloud. Choose the same region. Enter a globally unique registry name using lowercase letters and numbers only, such as hellocloudacr1234. Choose Basic, or the lowest suitable tier shown for your subscription.
3. Select Review + create, wait for validation, and select Create. When deployment finishes, select Go to resource.
4. On the registry Overview page, record the Login server, which ends with azurecr.io. After the image is pushed, open Repositories and select hello-cloud-app to verify the v2 tag.
5. The portal creates and displays the registry, but it does not upload the local Docker image by itself. Use the PowerShell block below, Docker commands in Cloud Shell, or an equivalent CI/CD workflow to push the image.

Visual checkpoint. Capture the registry Overview or Repositories page for Screenshot space 9.

Optional PowerShell route

```
az acr create --resource-group $RESOURCE_GROUP --name $ACR_NAME --sku Basic
$ACR_LOGIN_SERVER = az acr show --name $ACR_NAME --resource-group $RESOURCE_GROUP --query loginServer --output
tsv
az acr login --name $ACR_NAME

docker tag hello-cloud-app:v2 "$ACR_LOGIN_SERVER/hello-cloud-app:v2"
docker push "$ACR_LOGIN_SERVER/hello-cloud-app:v2"
az acr repository list --name $ACR_NAME --output table
```

Expected result: the repository list contains hello-cloud-app and the v2 tag.

Screenshot space 9 Show the ACR Overview or Repositories page with the v2 image tag

```
C:\student-cloud-lab>az acr repository list --name hellocloudacr1234 --output table
Result
-----
hello-cloud-app
C:\student-cloud-lab>az acr login --name hellocloudacr1234
Login Succeeded

C:\student-cloud-lab>docker tag hello-cloud-app:v2 hellocloudacr1234.azurecr.io/hello-cloud-app:v2

C:\student-cloud-lab>docker push hellocloudacr1234.azurecr.io/hello-cloud-app:v2
The push refers to repository [hellocloudacr1234.azurecr.io/hello-cloud-app]
a974ee19ff28: Pushed
70625f5e47a9: Pushed
6b37362b3da7: Pushed
16e00ab91de8: Pushed
b0dc7f87bef1: Pushed
3764a9a7d1e8: Pushed
b593643f1d10: Pushed
d9f3af818582: Pushed
06ad939ed42b: Pushed
v2: digest: sha256:803303402189dd45048b8096be79da38140da8dc21f4b35774928ceef676da6d size: 856
```

Step 11 Create the Azure Kubernetes Cluster

The following PowerShell route creates a small AKS cluster with one node and connects it to ACR. The portal route above performs the same setup visually. Cluster creation can take several minutes, and a node can consume student credit even when the application is very small.

Visual Azure Portal Path

Use the portal to create the AKS cluster. Keep the cluster small because this is a student-credit lab.

1. In the portal search bar, open Kubernetes services, select Create, and choose Kubernetes cluster.
2. On Basics, select the student subscription, rg-hello-cloud, and the chosen region. Set the cluster name to hello-cloud-aks. If shown, choose the Dev/Test preset and the Free AKS pricing tier.
3. On Node pools, keep one Linux node and set the node count to 1. Leave availability zones disabled for this small exercise, and review the displayed VM size and cost.
4. If the create form includes an Integrations tab, select the hellocloudacr registry so AKS can pull the private image. If that option is not available, use the existing --attach-acr PowerShell route below.
5. Select Review + create, wait for validation, and select Create. When deployment finishes, select Go to resource.
6. On the AKS Overview page, confirm that provisioning succeeded. Select Connect and follow the displayed Cloud Shell or Run command instructions to connect kubectl.
7. Open Kubernetes resources > Nodes and confirm that the node status is Ready.

Visual checkpoint. Capture the AKS Overview and Kubernetes resources Nodes page for Screenshot space 10.

Optional PowerShell route

```
az aks create `
  --resource-group $RESOURCE_GROUP `
  --name $AKS_NAME `
  --node-count 1 `
  --enable-managed-identity `
  --attach-acr $ACR_NAME `
  --generate-ssh-keys
```

Connect kubectl to AKS

Optional PowerShell route

```
az aks get-credentials --resource-group $RESOURCE_GROUP --name $AKS_NAME --overwrite-existing
kubectl config current-context
kubectl get nodes
```

Expected result: kubectl is connected to the AKS cluster and one node is Ready. Do not run the local Kubernetes commands until you switch the context back to docker-desktop.

Screenshot space 10 Show the AKS Overview and a Ready node in Kubernetes resources

```
C:\student-cloud-lab>az aks get-credentials --resource-group rg-hello-cloud --name hello-cloud-aks --overwrite-existing
Merged "hello-cloud-aks" as current context in C:\Users\Admin\.kube\config

C:\student-cloud-lab>kubectl config current-context
hello-cloud-aks

C:\student-cloud-lab>kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-nodepool1-21211696-vmss000000  Ready    <none>   5m31s  v1.35.8

C:\student-cloud-lab>|
```

Step 12 Deploy the Image to Azure AKS

Create a second manifest for Azure. The only important image change is that the image now comes from ACR. The Service type is LoadBalancer so Azure can provide a public IP address.

File Five Azure Kubernetes Configuration

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-cloud-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-cloud-app
  template:
    metadata:
      labels:
        app: hello-cloud-app
    spec:
      containers:
        - name: hello-cloud-app
          image: ACR_LOGIN_SERVER/hello-cloud-app:v2
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: hello-cloud-app
spec:
  selector:
    app: hello-cloud-app
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 5000
```

Replace the text ACR_LOGIN_SERVER with the value printed by the PowerShell variable. For example, the image line will look like this after replacement:

YAML example

```
image: hellocloudacr1234.azurecr.io/hello-cloud-app:v2
```

Visual Azure Portal Deployment Path

After replacing ACR_LOGIN_SERVER in File Five with the real login server, apply the same YAML from the browser.

1. Open the AKS cluster in the Azure portal. Under Kubernetes resources, select Services and ingresses.

2. Select Create > Apply a YAML.
3. Paste the complete content of k8s-azure.yaml into the editor. If the portal shows a Next or Review step, continue to the review screen.
4. Review the hello-cloud-app Deployment and the LoadBalancer Service, then select Deploy and Close or Apply, depending on the portal view.
5. Open Workloads > Deployments and confirm that hello-cloud-app has two ready replicas. Open Services and ingresses, select hello-cloud-app, and wait for an External IP.
6. Open the External IP in a browser and add /health to test the health endpoint.

If Apply a YAML is not visible. Use AKS > Overview > Connect to open Cloud Shell or use the existing PowerShell route below.

Optional PowerShell shortcut: replace the placeholder automatically before applying the file.

PowerShell

```
(Get-Content .\k8s-azure.yaml) -replace 'ACR_LOGIN_SERVER', $ACR_LOGIN_SERVER | Set-Content .\k8s-azure.yaml
kubectl apply -f .\k8s-azure.yaml
kubectl rollout status deployment/hello-cloud-app
kubectl get pods
kubectl get service hello-cloud-app
```

The External IP may remain pending for a few minutes. Watch it until an IP address appears.

Optional PowerShell route

```
kubectl get service hello-cloud-app --watch
```

Press Ctrl plus C after the external IP appears. Open <http://EXTERNAL-IP> in a browser and add /health to test the health endpoint.

Screenshot space 11 Show Services and ingresses with the External IP and the app running in a browser

Insert screenshot here

Step 13 Update the Azure Application

Build and push version three with Docker or Cloud Shell as shown below. Then update the Kubernetes image through the Azure portal or the PowerShell route.

Change the App to Version Three

Python change

```
APP_VERSION = "v3"

# Use a new message inside home
message="Version three is running on Azure"
```

Build and Push the New Image

Optional PowerShell route

```
docker build -t hello-cloud-app:v3 .
docker tag hello-cloud-app:v3 "$ACR_LOGIN_SERVER/hello-cloud-app:v3"
docker push "$ACR_LOGIN_SERVER/hello-cloud-app:v3"
```

Tell Kubernetes to Use Version Three

Optional PowerShell route

```
kubectl set image deployment/hello-cloud-app `
  hello-cloud-app="$ACR_LOGIN_SERVER/hello-cloud-app:v3"
kubectl rollout status deployment/hello-cloud-app
kubectl get pods -o wide
```

Refresh the Azure public IP address in the browser. Kubernetes keeps the Service address stable while it replaces the old pods with the new image. If the new pods fail their health check, Kubernetes does not consider them ready.

Visual Azure Portal Update Path

After the v3 image is pushed to ACR, update the image reference in the AKS portal.

1. Open AKS > Kubernetes resources > Workloads > Deployments and select hello-cloud-app.
2. Open YAML or Edit YAML, depending on the portal view. Find the container image ending in :v2 and change only the tag to :v3. Keep the ACR login server and image name unchanged.
3. Select Review, Save, or Apply. Wait until the Deployment shows two ready replicas.
4. Open Pods to observe the replacement pods. Open Services and ingresses to use the same External IP, then refresh the browser and confirm that version three is running.
5. Capture the Deployment or Pods view and the browser response for Screenshot space 12.

Fallback portal method. If direct editing is not available, use Services and ingresses > Create > Apply a YAML and paste the updated manifest with the v3 image.

Screenshot space 12 Show the portal workload status, v3 pods, and the version three response

Insert screenshot here

Rollback the Azure Version

Optional PowerShell route

```
kubectl rollout history deployment/hello-cloud-app
kubectl rollout undo deployment/hello-cloud-app
kubectl rollout status deployment/hello-cloud-app
```

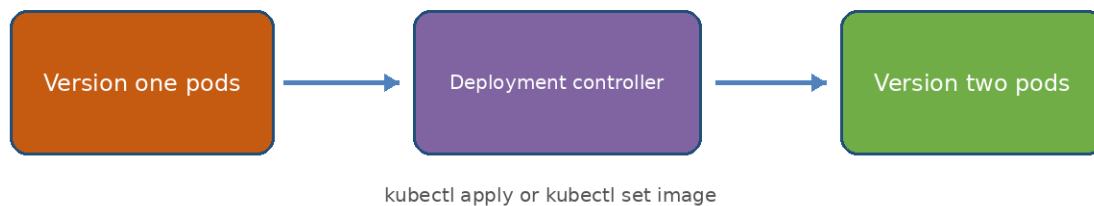
Test the same public IP again. The previous image should be running. This shows how Kubernetes can reduce the risk of a bad update.

Visual Portal Rollback

1. Open Workloads > Deployments > hello-cloud-app and open YAML or Edit YAML.
 2. Change the image tag from :v3 back to :v2 and apply the manifest.
 3. Wait for the previous version's pods to become Ready, then refresh the same public IP in the browser.
- Revision history note.** The portal view may not expose revision-based rollout undo in every cluster view. Use the existing kubectl rollout undo command when you need Kubernetes revision history.

How Kubernetes Helps with Updates

Rolling update: new pods become ready before old pods disappear



The Deployment controller manages the transition from v1 to v2 or v3

Without Kubernetes	With Kubernetes
The developer may stop the only running container.	Multiple replicas can keep serving traffic.
The developer must manage the replacement manually.	The Deployment controller manages replacement pods.
A failed update may require rebuilding the environment.	rollout undo can return to a previous revision.
The service address may change when a process changes.	The Service keeps a stable endpoint.
Health checks are often separate scripts.	Readiness and liveness probes are part of the workload configuration.

Kubernetes does not remove the need for testing. It provides a controlled way to run, observe, scale, update, and roll back the application after a tested image is available.

Troubleshooting Guide

Message or symptom	What to check
docker is not recognised	Restart PowerShell after installing Docker Desktop. Confirm Docker Desktop is running.
Cannot connect to Docker daemon	Open Docker Desktop and wait until the engine is ready.
kubectl get nodes fails locally	Enable Kubernetes in Docker Desktop and use kubectl config use-context docker-desktop.

Message or symptom	What to check
ImagePullBackOff locally	Confirm the image name and tag. Use imagePullPolicy IfNotPresent for the local image.
CrashLoopBackOff	Run <code>kubectl logs deployment/hello-cloud-app</code> and check the Python error.
ACR name error	Use only lowercase letters and numbers and choose a globally unique name.
AKS quota or capacity error	Try another region or delete unused resources in the student subscription.
External IP is pending	Wait, then run <code>kubectl describe service hello-cloud-app</code> and check service events.
Azure credits are being consumed	Stop the cluster when finished and delete the resource group.

Useful Diagnostic Commands

PowerShell

```
kubectl get pods
kubectl describe pod <POD_NAME>
kubectl logs deployment/hello-cloud-app
kubectl get events --sort-by=.lastTimestamp
docker ps
docker logs hello-cloud-app
```

Screenshot space 13 Record one portal diagnostic view or troubleshooting command and its result

Insert screenshot here

Step 14 Clean Up Azure Resources

Delete the resource group through the Azure portal or PowerShell when the class exercise is finished. This removes the ACR and AKS resources created in this practical. Do not delete anything until students have saved their screenshots and results.

Visual Azure Portal Cleanup

1. In the Azure portal search bar, open Resource groups and select rg-hello-cloud.
2. Select Delete resource group.
3. Enter rg-hello-cloud, confirm the deletion, and select Delete again.
4. Wait until the resource group disappears. If required, open Cost Management + Billing or the subscription cost view to check recent usage.

Important. Do not delete the group until all screenshots and student results have been saved.

Optional PowerShell route

```
az group delete --name $RESOURCE_GROUP --yes --no-wait  
az group show --name $RESOURCE_GROUP
```

The final command may report that the resource group is not found after deletion is complete. That is expected. You can also verify usage in the Azure portal under Cost Management.

Screenshot space 14 Show the portal deletion confirmation or the deleted resource group

Insert screenshot here

Student Submission Checklist

- ☐ A screenshot of the Flask application running without Docker.
- ☐ A screenshot of the Docker image, running container, and browser response.
- ☐ A screenshot of the local Kubernetes pods and NodePort service.
- ☐ A screenshot showing the local update from v1 to v2.
- ☐ A screenshot of the ACR Overview or Repositories page showing the image tag.
- ☐ A screenshot of the AKS portal Nodes and Services pages showing a Ready node and External IP.
- ☐ A screenshot showing the Azure update from v2 to v3 in the portal or through kubectl.
- ☐ A screenshot of a successful portal deployment or rollback view, or a rollout command.
- ☐ A short explanation of how replicas, Services, probes, rolling updates, and rollback help.

Reflection Questions

1. Why did we test the application without Docker first?

Answer:

2. What is the difference between a Docker image and a running container?

Answer:

3. Why does the Kubernetes Deployment use replicas equal to two?

Answer:

4. What does the Service provide that a pod does not provide?

Answer:

5. Why do we create a new image tag for each version?

Answer:

6. What happens if the new version fails its health check?

Answer:

7. Why must we delete the Azure resource group after the lab?

Answer:

Official References

- [Docker Desktop Kubernetes](#)
- [Azure CLI for Windows](#)
- [Azure Container Registry Docker guide](#)
- [Create an AKS cluster with Azure CLI](#)
- [Azure for Students](#)
- [Track Azure for Students costs](#)
- [Create an Azure Container Registry in the Azure portal](#)
- [Create an AKS cluster using the Azure portal](#)
- [Manage Kubernetes resources in the Azure portal](#)